Introduction to Theoretical Computer Science

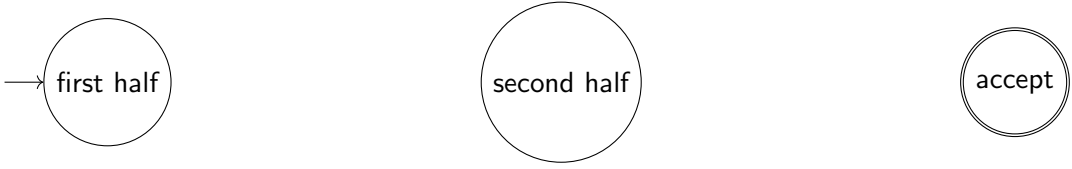
CS2490 - Session 27: Pushdown Automata and Context-Free Languages

Pierre Cagne

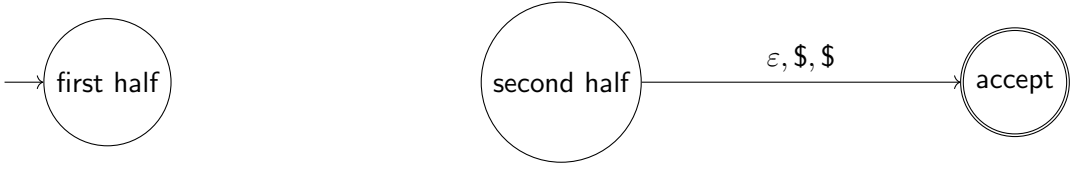
Fall 2026

October 21, 2026

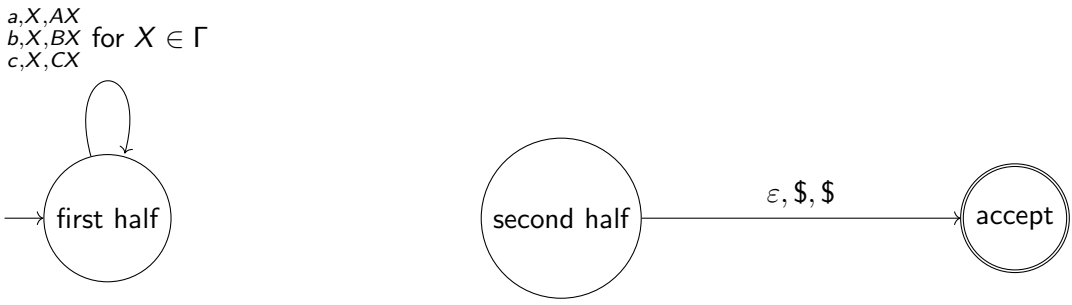
A PDA for palindromes



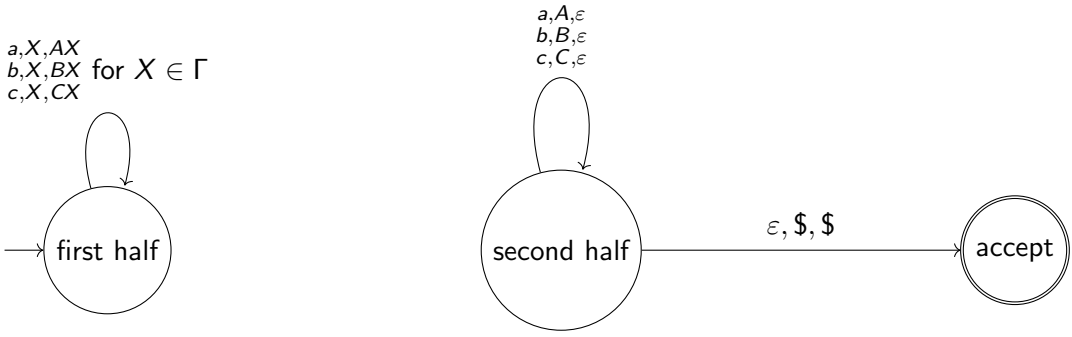
A PDA for palindromes



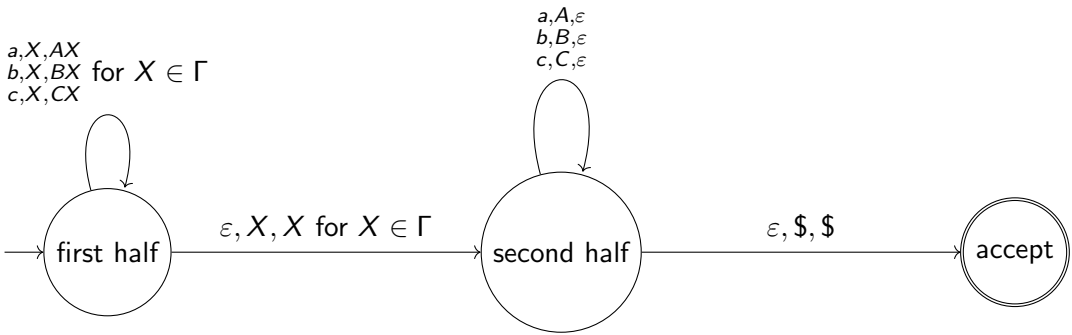
A PDA for palindromes



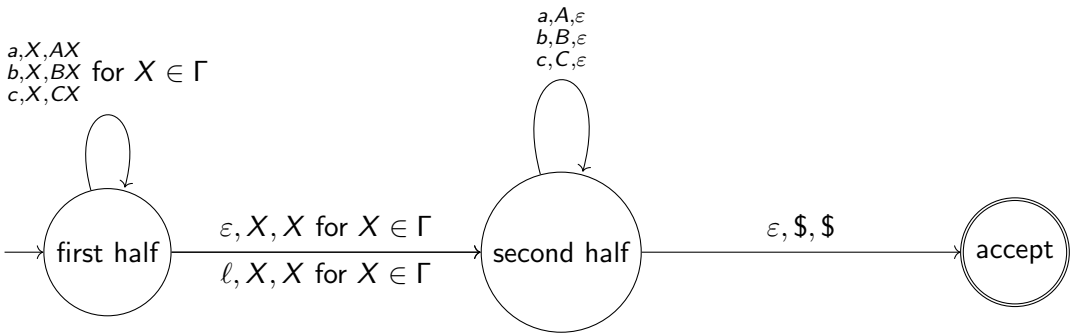
A PDA for palindromes



A PDA for palindromes



A PDA for palindromes



A PDA for palindromes

Wait!

A PDA for palindromes

Wait! How are we sure the computation switches from the first half to the second half at the right moment?

A PDA for palindromes

Wait! How are we sure the computation switches from the first half to the second half at the right moment?

We aren't! That's the beauty of non-determinism: one of the possible computations will jump at the right time, and that's all that matters.

A PDA for palindromes

Wait! How are we sure the computation switches from the first half to the second half at the right moment?

We aren't! That's the beauty of non-determinism: one of the possible computations will jump at the right time, and that's all that matters.

Let's see it in action on the word abbcabacbba.

A puzzling question

Question: drawing inspiration from the previous example, can we design a PDA that recognizes the language

$$L = \{ww \mid w \in \{a, b, c\}^*\} \quad ?$$

A puzzling question

Question: drawing inspiration from the previous example, can we design a PDA that recognizes the language

$$L = \{ww \mid w \in \{a, b, c\}^*\} \quad ?$$

Answer: NO!

A puzzling question

Question: drawing inspiration from the previous example, can we design a PDA that recognizes the language

$$L = \{ww \mid w \in \{a, b, c\}^*\} \quad ?$$

Answer: NO! The informal reason is that leaving clues while reading the first half won't help, since we would have to pop them in reverse order while reading the second half.

A puzzling question

Question: drawing inspiration from the previous example, can we design a PDA that recognizes the language

$$L = \{ww \mid w \in \{a, b, c\}^*\} \quad ?$$

Answer: NO! The informal reason is that leaving clues while reading the first half won't help, since we would have to pop them in reverse order while reading the second half. This is not a proof though, since we could imagine a more clever PDA, not based on the same reasoning as before.

A puzzling question

Question: drawing inspiration from the previous example, can we design a PDA that recognizes the language

$$L = \{ww \mid w \in \{a, b, c\}^*\} \quad ?$$

Answer: NO! The informal reason is that leaving clues while reading the first half won't help, since we would have to pop them in reverse order while reading the second half. This is not a proof though, since we could imagine a more clever PDA, not based on the same reasoning as before.

We will see an actual proof later that this language cannot be recognized by any PDA.

One more!

Let's design a PDA that recognizes the language $L = \{a^n b^{3n} \mid n \geq 0\}$.

One more!

Let's design a PDA that recognizes the language $L = \{a^n b^{3n} \mid n \geq 0\}$.

The idea: when reading an a, push a clue onto the stack; when reading a b, pop a clue from the stack but only once every three bs.

One more!

Let's design a PDA that recognizes the language $L = \{a^n b^{3n} \mid n \geq 0\}$.

The idea: when reading an a, push a clue onto the stack; when reading a b, pop a clue from the stack but only once every three bs.

Thus, beside the initial stack symbol \$, one stack symbol $|$.

One more!

Let's design a PDA that recognizes the language $L = \{a^n b^{3n} \mid n \geq 0\}$.

The idea: when reading an a, push a clue onto the stack; when reading a b, pop a clue from the stack but only once every three bs.

Thus, beside the initial stack symbol \$, one stack symbol $|$.

How many states?

One more!

Let's design a PDA that recognizes the language $L = \{a^n b^{3n} \mid n \geq 0\}$.

The idea: when reading an a, push a clue onto the stack; when reading a b, pop a clue from the stack but only once every three bs.

Thus, beside the initial stack symbol \$, one stack symbol $|$.

How many states? One to read all the as

One more!

Let's design a PDA that recognizes the language $L = \{a^n b^{3n} \mid n \geq 0\}$.

The idea: when reading an a, push a clue onto the stack; when reading a b, pop a clue from the stack but only once every three bs.

Thus, beside the initial stack symbol \$, one stack symbol $|$.

How many states? One to read all the as and three to read the bs.

One more!

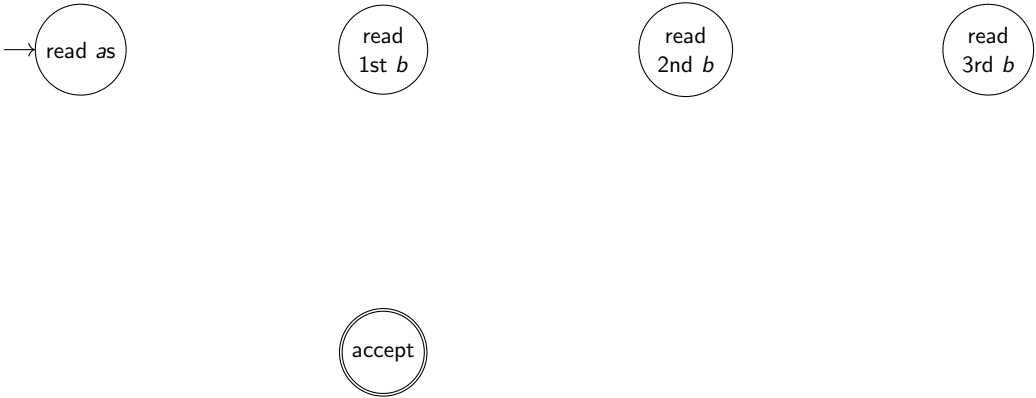
Let's design a PDA that recognizes the language $L = \{a^n b^{3n} \mid n \geq 0\}$.

The idea: when reading an a, push a clue onto the stack; when reading a b, pop a clue from the stack but only once every three bs.

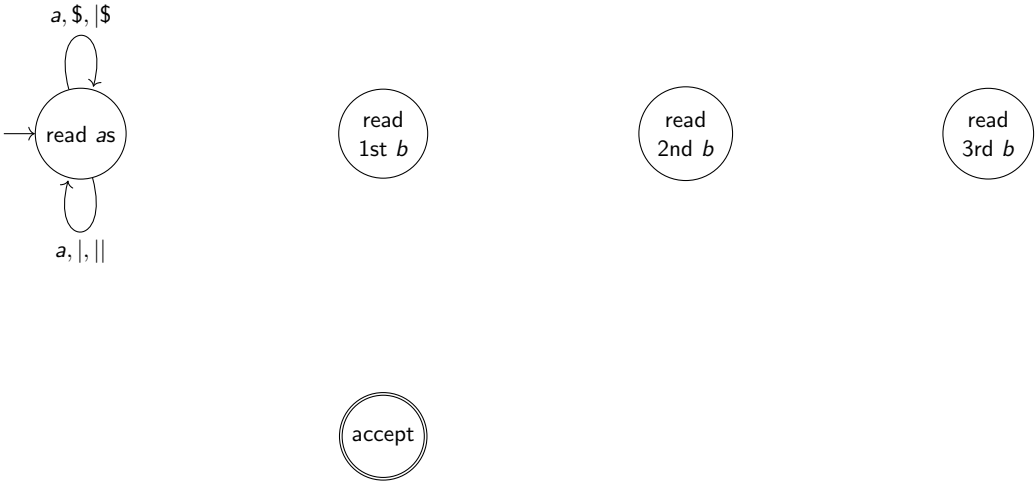
Thus, beside the initial stack symbol \$, one stack symbol $|$.

How many states? One to read all the as and three to read the bs. Plus one to accept.

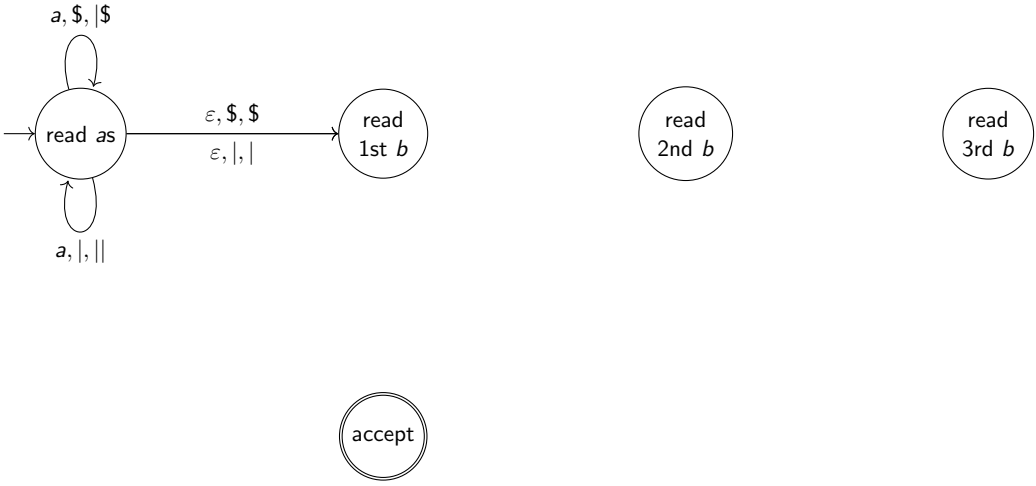
One more! (cont'd)



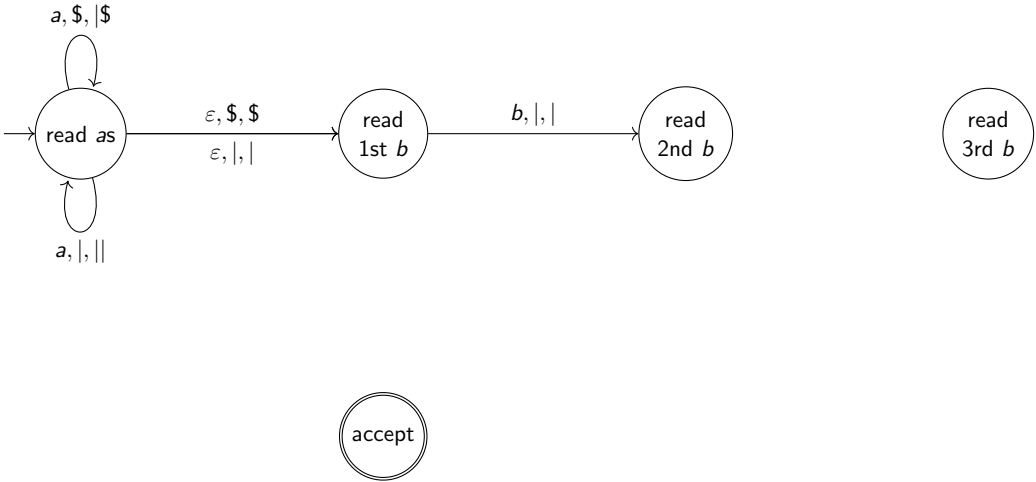
One more! (cont'd)



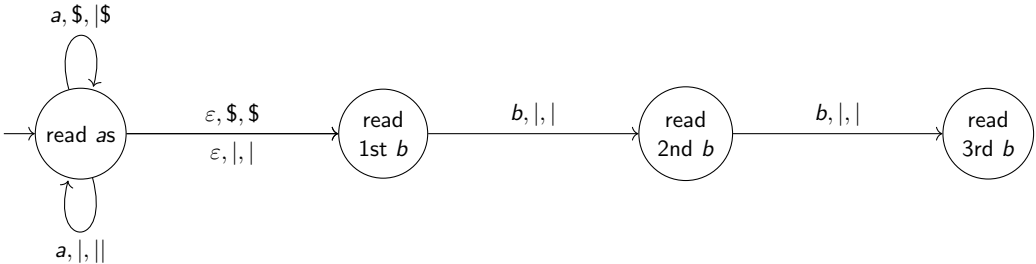
One more! (cont'd)



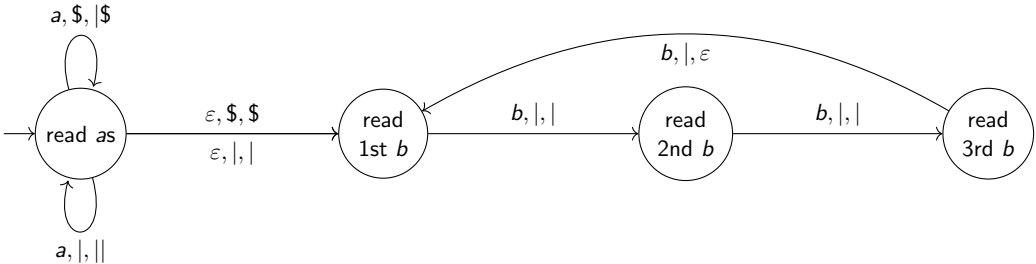
One more! (cont'd)



One more! (cont'd)

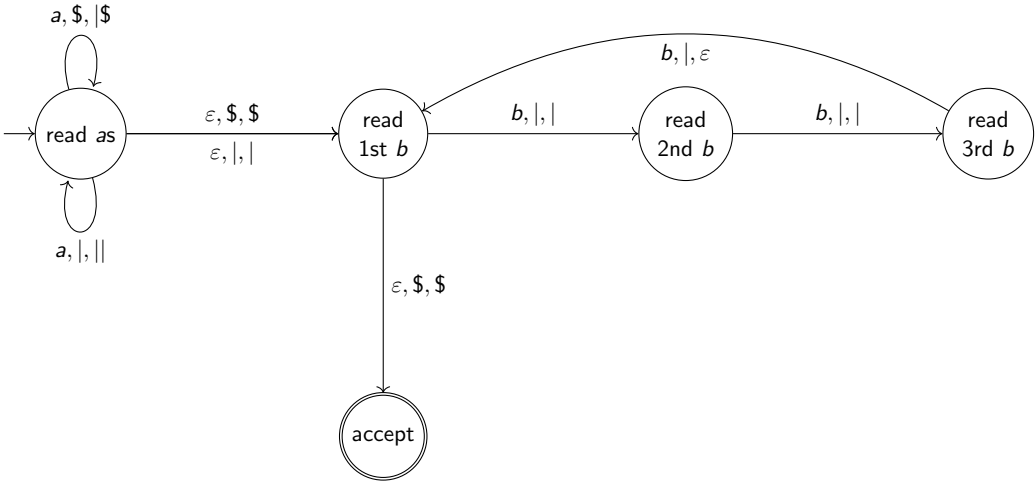


One more! (cont'd)

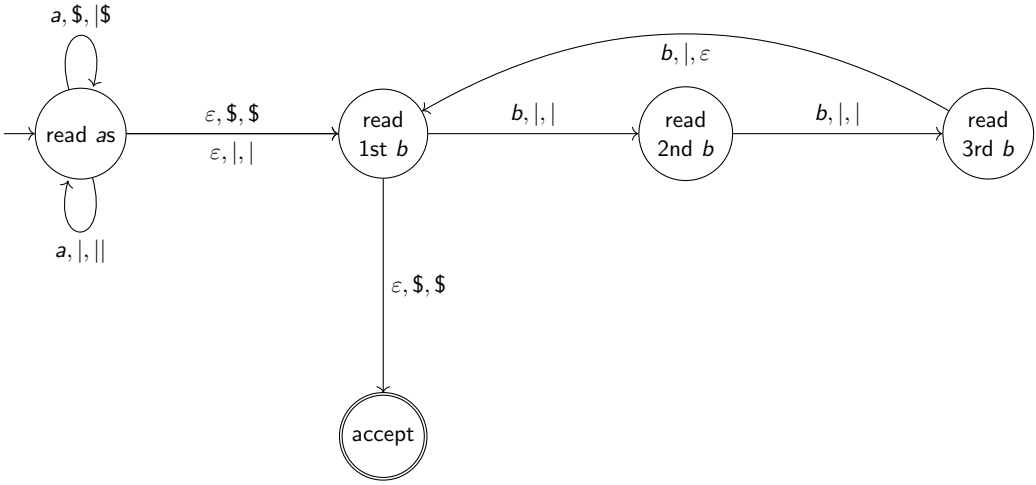




One more! (cont'd)



One more! (cont'd)



Let's see it in action on the word aaabbbbbbbbb.

PDA vs CFL

Remember we have introduced PDA by claiming they would be to CFL what DFA/NFA are to regular languages.

PDA vs CFL

Remember we have introduced PDA by claiming they would be to CFL what DFA/NFA are to regular languages.

Indeed:

Theorem

A language is context-free if and only if it is recognized by a PDA.

PDA vs CFL

Remember we have introduced PDA by claiming they would be to CFL what DFA/NFA are to regular languages.

Indeed:

Theorem
A language is context-free if and only if it is recognized by a PDA.

Today, we are going to see an algorithm that justifies the direction $(\implies)$.

PDA vs CFL

Remember we have introduced PDA by claiming they would be to CFL what DFA/NFA are to regular languages.

Indeed:

Theorem
A language is context-free if and only if it is recognized by a PDA.

Today, we are going to see an algorithm that justifies the direction ($\Longrightarrow$). In other words, given a CFG G, we are going to see how to design a PDA that recognizes the language $\mathrm{L}(G)$ generated by G.

Modifying CFGs

Claim: Any CFG G can be transformed into an equivalent CFG $G' = (\Sigma, V', S', P')$ whose production rules are all of the form:

- $A \to a$ where $A \in V'$ and $a \in \Sigma \cup \{\varepsilon\}$, or
- $A \to W$ where $A \in V'$ and $W \in V'^*$ (only variable symbols on the right-hand side).

Modifying CFGs

Claim: Any CFG G can be transformed into an equivalent CFG $G' = (\Sigma, V', S', P')$ whose production rules are all of the form:

- $A \to a$ where $A \in V'$ and $a \in \Sigma \cup \{\varepsilon\}$, or
- $A \to W$ where $A \in V'$ and $W \in V'^*$ (only variable symbols on the right-hand side).

In this class, such a G' will be said to be in *Generalized Chomsky Normal Form*. This is not standard terminology!

Modifying CFGs (cont'd)

How? Given a CFG $G = (\Sigma, V, S, P)$, construct $G' = (\Sigma, V', S', P')$ as follows:

Modifying CFGs (cont'd)

How? Given a CFG $G = (\Sigma, V, S, P)$, construct $G' = (\Sigma, V', S', P')$ as follows:

- Σ is of course the same,

Modifying CFGs (cont'd)

How? Given a CFG $G = (\Sigma, V, S, P)$, construct $G' = (\Sigma, V', S', P')$ as follows:

- Σ is of course the same,
- V' contains every variable of V

Modifying CFGs (cont'd)

How? Given a CFG $G = (\Sigma, V, S, P)$, construct $G' = (\Sigma, V', S', P')$ as follows:

- Σ is of course the same,
- V' contains every variable of V plus one new variable A_a per letter $a \in \Sigma$,

Modifying CFGs (cont'd)

How? Given a CFG $G = (\Sigma, V, S, P)$, construct $G' = (\Sigma, V', S', P')$ as follows:

- Σ is of course the same,
- V' contains every variable of V plus one new variable A_a per letter $a \in \Sigma$,
- $S' = S$,

Modifying CFGs (cont'd)

How? Given a CFG $G = (\Sigma, V, S, P)$, construct $G' = (\Sigma, V', S', P')$ as follows:

- Σ is of course the same,
- V' contains every variable of V plus one new variable A_a per letter $a \in \Sigma$,
- $S' = S$,
- P' is obtained by copying every production rule from P in which we replace each occurrence of the letter $a \in \Sigma$ by the corresponding variable A_a.

Modifying CFGs (cont'd)

How? Given a CFG $G = (\Sigma, V, S, P)$, construct $G' = (\Sigma, V', S', P')$ as follows:

- Σ is of course the same,
- V' contains every variable of V plus one new variable A_a per letter $a \in \Sigma$,
- $S' = S$,
- P' is obtained by copying every production rule from P in which we replace each occurrence of the letter $a \in \Sigma$ by the corresponding variable A_a. Then, we also add the production rules

$$A_a \to a \quad \text{for each } a \in \Sigma.$$

Modifying CFGs (cont'd)

How? Given a CFG $G = (\Sigma, V, S, P)$, construct $G' = (\Sigma, V', S', P')$ as follows:

- Σ is of course the same,
- V' contains every variable of V plus one new variable A_a per letter $a \in \Sigma$,
- $S' = S$,
- P' is obtained by copying every production rule from P in which we replace each occurrence of the letter $a \in \Sigma$ by the corresponding variable A_a. Then, we also add the production rules

$$A_a \to a \quad \text{for each } a \in \Sigma.$$

It is easy to see that G' generates the same language as G.

Example

Let's transform the CFG $G = (\{a, b\}, \{S\}, S, P)$ with production rules:

$$S \to aSa \mid bSb \mid a \mid b \mid \varepsilon$$

into an equivalent CFG G' in Generalized Chomsky Normal Form.

Example

Let's transform the CFG $G = (\{a, b\}, \{S\}, S, P)$ with production rules:

$$S \to aSa \mid bSb \mid a \mid b \mid \varepsilon$$

into an equivalent CFG G' in Generalized Chomsky Normal Form.

$$S \to aSa \mid bSb \mid a \mid b \mid \varepsilon$$

Example

Let's transform the CFG $G = (\{a, b\}, \{S\}, S, P)$ with production rules:

$$S \to aSa \mid bSb \mid a \mid b \mid \varepsilon$$

into an equivalent CFG G' in Generalized Chomsky Normal Form.

$$S \to A_a S A_a \mid A_b S A_b \mid A_a \mid A_b \mid \varepsilon$$

Example

Let's transform the CFG $G = (\{a, b\}, \{S\}, S, P)$ with production rules:

$$S \to aSa \mid bSb \mid a \mid b \mid \varepsilon$$

into an equivalent CFG G' in Generalized Chomsky Normal Form.

$$S \to A_aSA_a \mid A_bSA_b \mid A_a \mid A_b \mid \varepsilon \quad A_a \to a \quad A_b \to b$$

Transforming a CFG into a PDA

Let $G = (\Sigma, V, S, P)$ be a CFG.

Transforming a CFG into a PDA

Let $G = (\Sigma, V, S, P)$ be a CFG. WLOG, we can assume that G is in Generalized Chomsky Normal Form.

Transforming a CFG into a PDA

Let $G = (\Sigma, V, S, P)$ be a CFG. WLOG, we can assume that G is in Generalized Chomsky Normal Form. *(If it is not, transform it using the method just seen.)*

Transforming a CFG into a PDA

Let $G = (\Sigma, V, S, P)$ be a CFG. WLOG, we can assume that G is in Generalized Chomsky Normal Form. *(If it is not, transform it using the method just seen.)*

The idea is to use the stack to simulate the leftmost derivations of G.

Transforming a CFG into a PDA

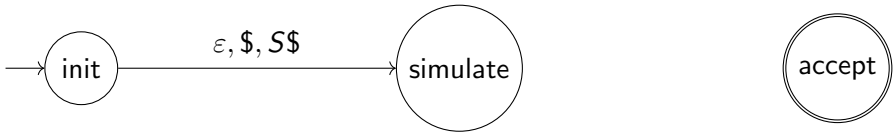
Let $G = (\Sigma, V, S, P)$ be a CFG. WLOG, we can assume that G is in Generalized Chomsky Normal Form. *(If it is not, transform it using the method just seen.)*

The idea is to use the stack to simulate the leftmost derivations of G. Thus, the stack alphabet will be $\Gamma = V \cup \{\$\}$, where \$ is the initial stack symbol.

Transforming a CFG into a PDA

Let $G = (\Sigma, V, S, P)$ be a CFG. WLOG, we can assume that G is in Generalized Chomsky Normal Form. *(If it is not, transform it using the method just seen.)*

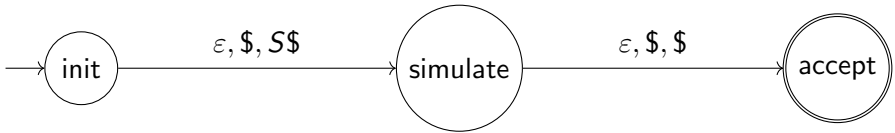
The idea is to use the stack to simulate the leftmost derivations of G. Thus, the stack alphabet will be $\Gamma = V \cup \{\$\}$, where \$ is the initial stack symbol.



Transforming a CFG into a PDA

Let $G = (\Sigma, V, S, P)$ be a CFG. WLOG, we can assume that G is in Generalized Chomsky Normal Form. *(If it is not, transform it using the method just seen.)*

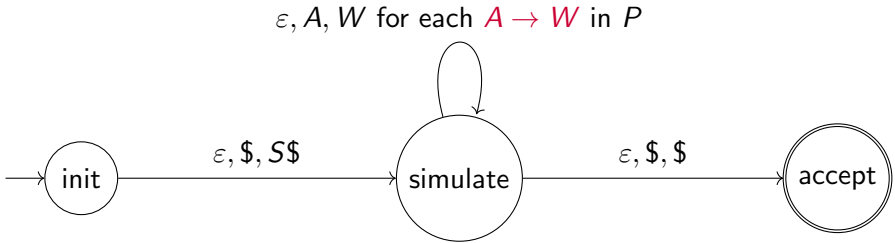
The idea is to use the stack to simulate the leftmost derivations of G. Thus, the stack alphabet will be $\Gamma = V \cup \{\$\}$, where \$ is the initial stack symbol.



Transforming a CFG into a PDA

Let $G = (\Sigma, V, S, P)$ be a CFG. WLOG, we can assume that G is in Generalized Chomsky Normal Form. *(If it is not, transform it using the method just seen.)*

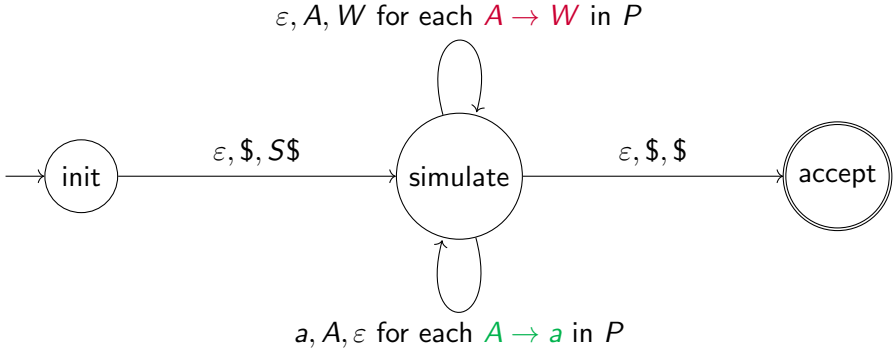
The idea is to use the stack to simulate the leftmost derivations of G. Thus, the stack alphabet will be $\Gamma = V \cup \{\$\}$, where \$ is the initial stack symbol.



Transforming a CFG into a PDA

Let $G = (\Sigma, V, S, P)$ be a CFG. WLOG, we can assume that G is in Generalized Chomsky Normal Form. *(If it is not, transform it using the method just seen.)*

The idea is to use the stack to simulate the leftmost derivations of G. Thus, the stack alphabet will be $\Gamma = V \cup \{\$\}$, where \$ is the initial stack symbol.



Example

Let's do it on the CFG for palindromes over $\{a, b\}$ seen earlier!

Example

Let's do it on the CFG for palindromes over $\{a, b\}$ seen earlier!

We already have the GCNF:

$$S \to A_a S A_a \mid A_b S A_b \mid A_a \mid A_b \mid \varepsilon \quad A_a \to a \quad A_b \to b$$

Example

Let's do it on the CFG for palindromes over $\{a, b\}$ seen earlier!
We already have the GCNF:

$$S \to A_aSA_a \mid A_bSA_b \mid A_a \mid A_b \mid \varepsilon \quad A_a \to a \quad A_b \to b$$

Thus, the PDA is:

